

# Chapitre 4:

## Système de Gestion de Fichiers

### Cours 5: Généralités sur les fichiers

# Plan

- 1) Organisation des données sur le disque**
- 2) Notion de fichier**
- 3) Types de fichier**
- 4) Système de fichier**
- 5) Implémentation du Système de fichier**
- 6) Notion d'i-node**

# Plan

- 1) Organisation des données sur le disque**
- 2) Notion de fichier
- 3) Types de fichier
- 4) Système de fichier
- 5) Implémentation du Système de fichier
- 6) Notion d'i-node

# Organisation des données sur le disque

## Piste (Track)

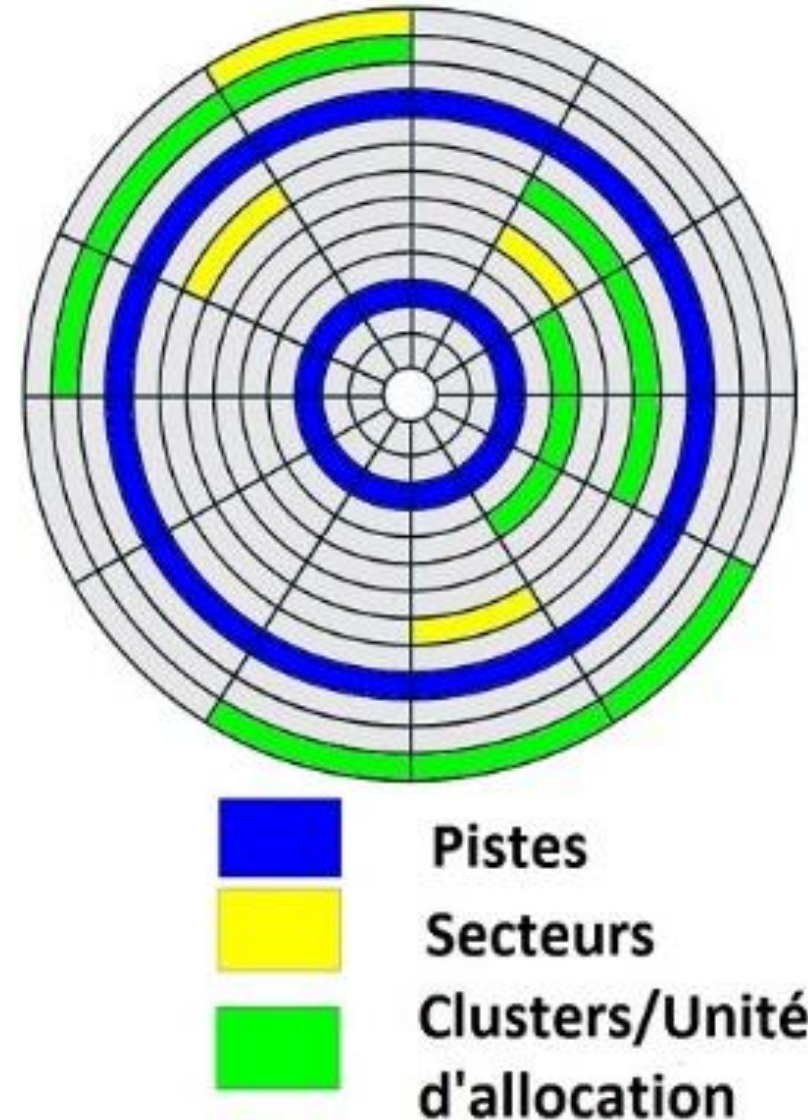
- Cercle concentrique sur la surface d'un disque.
- Contient plusieurs secteurs.

## Secteur (Sector)

- Plus petite unité physique de stockage sur un disque (ex : 512 octets ou 4 Ko).
- Une piste est divisée en secteurs.

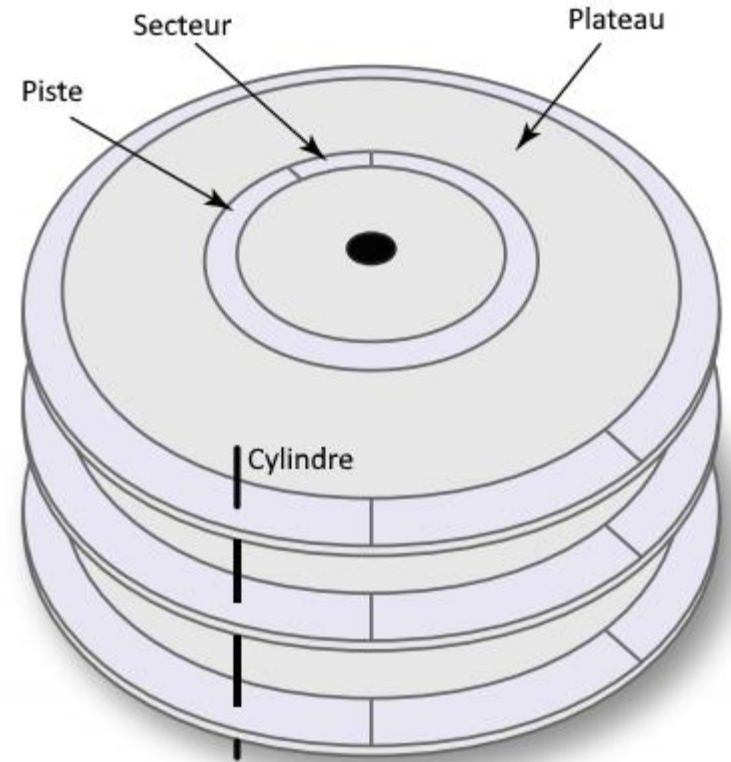
## Cluster (ou Allocation Unit / Bloc)

- Plus petite unité logique de stockage du SF.
- Regroupe un ou plusieurs secteurs.
- Le SE lit/écrit les données par clusters (et non par secteurs).
- Un fichier occupe au moins un cluster (même s'il est plus petit).



# Organisation des données sur le disque

- **Vue physique** : pour accéder à un secteur, il faut connaître le cylindre, le plateau et le secteur: très fastidieux !
- **Vue logique** : le SF fournit une vue logique du disque permettant de: Faire abstraction des propriétés physiques des dispositifs de stockage.



# Plan

- 1) Organisation des données sur le disque
- 2) Notion de fichier**
- 3) Types de fichier
- 4) Système de fichier
- 5) Implémentation du Système de fichier
- 6) Notion d'i-node

# Notion de fichier

## Définition de fichier

- Un **fichier** est une unité logique d'information stockée sur un support mémoire (disque, SSD, clé USB, etc.), identifiée par un nom et gérée par le système de fichiers.
- Il contient des données (texte, programme, image, audio...) et est enregistré sous forme de **blocs/clusters**, qui sont les unités d'allocation du système de fichiers.
- Un fichier occupe au minimum un bloc/cluster, même si sa taille est inférieure à celui-ci.

## Points clés

- Le **fichier** est une abstraction logique pour l'utilisateur.
- Il est **organisé physiquement** en secteurs, mais le système gère l'espace en **blocs/clusters**.
- Le fichier possède des **métadonnées** (nom, taille, type, dates, droits d'accès...).

# Notion de fichier

## Les attributs d'un fichier:

- Le nom,
- La taille, exprimée en octets,
- Les permission d'accès (Lecture, écriture, exécution),
- Le contenu du fichier (une suite d'octets),
- date de création et date de modification.
- Le nom du propriétaire du fichier.

# Notion de fichier

- Le nom du fichier est composé en général de deux parties séparées par un point: **nomfichier.extension**
  - La partie qui suit le point est appelée **extension** (prog.c, entete.h, fich.doc, toto.pdf ...).
  - L'extension est nécessaire dans certains cas. Par exemple, le compilateur C rejettera le fichier prog.txt même si son contenu est un programme C.
- Chaque fichier est identifié par une **référence** (un chemin d'accès) permettant de le localiser.

# Plan

- 1) Organisation des données sur le disque
- 2) Notion de fichier
- 3) Types de fichier**
- 4) Système de fichier
- 5) Implémentation du Système de fichier
- 6) Notion d'i-node

# Types de fichier

Un système d'exploitation gère plusieurs catégories de fichiers :

- **Fichiers ordinaires:** Contiennent des données ou des programmes. Ils peuvent être **texte (ASCII)** ou **binares**.
- **Répertoires:** Ce sont des fichiers spéciaux utilisés par le système pour organiser et structurer l'ensemble des fichiers.
- **Fichiers spéciaux caractère (Character devices):** Représentent les périphériques fonctionnant **en mode flux** (transmission caractère par caractère). Exemples : **claviers, terminaux, imprimantes, ports séries, interfaces réseau**.
- **Fichiers spéciaux bloc (Block devices):** Représentent les périphériques stockant et transférant les données **par blocs**. Exemples : **disques durs, SSD, disquettes, clés USB**.

Les fichiers **caractères** et **blocs** permettent au système d'interagir avec les périphériques comme s'il s'agissait de fichiers, facilitant ainsi les opérations d'E/S.

# Plan

- 1) Organisation des données sur le disque
- 2) Notion de fichier
- 3) Types de fichier
- 4) Système de fichier**
- 5) Implémentation du Système de fichier
- 6) Notion d'i-node

# Système de fichier (File System)

- Un **système de fichiers** (SF) est une structure logicielle qui organise et gère les données sur les **mémoires secondaires** (disque dur, SSD, CD-ROM, clé USB, etc.).
- Il permet de stocker les informations sous forme de **fichiers** et de les organiser efficacement.

## Rôle principal :

- Gérer les fichiers et leurs emplacements
- Fournir les opérations de base: **Créer, lire, écrire, modifier, supprimer** les fichiers

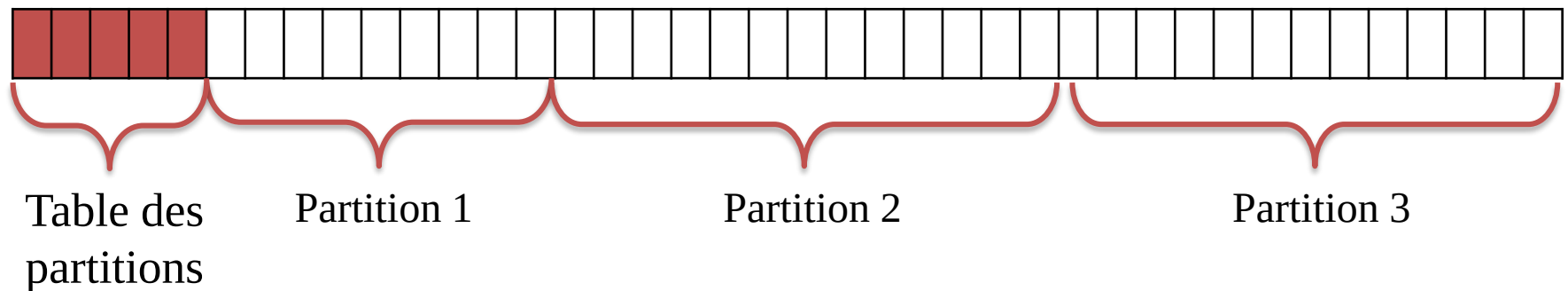
En résumé, le SF est l'intermédiaire qui permet au système d'exploitation et à l'utilisateur d'accéder aux données de manière structurée et fiable.

# Plan

- 1) Organisation des données sur le disque
- 2) Notion de fichier
- 3) Types de fichier
- 4) Système de fichier
- 5) Implémentation du Système de fichier**
- 6) Notion d'i-node

# Implémentation du Système de fichier

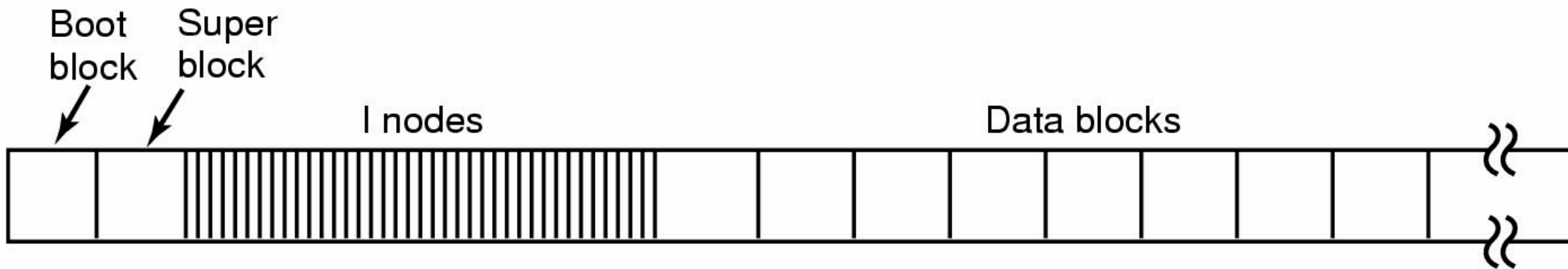
- Le disque est vue comme un long ruban constitué de plusieurs blocs de même taille 512 oct.
- Ce ruban est divisée en plusieurs parties de différentes tailles qu'on appelle **partition**.
- Chaque partition a un système de fichier.
- La table des partitions se trouve au début du disque, c'est un ensemble de blocs qui permettent de gérer les partitions du disque.



# Implémentation du Système de fichier

L'organisation physique d'une partition adoptant le système de fichiers EXT2/EXT3 (Linux) est la suivante:

- le bloc 0 est la zone de démarrage (« Boot Block »),
- le bloc 1 est le **super-bloc** qui contient les informations concernant la partition en question.
- Ensuite, vient la table des i-nodes,
- puis la zone des données.



# Implémentation du Système de fichier

- Avant qu'un SF puisse créer et gérer des fichiers sur une partition, cette dernière doit être **formatée** selon les spécificités du SF.
- Le **formatage** inspecte les secteurs, efface les données et crée le répertoire racine du système de fichiers.
- Il crée également un **super-bloc** pour stocker les informations nécessaires à assurer l'intégrité du SF tels que:
  1. L'identifiant du système de fichiers (C:, D : ..)
  2. Le nombre de blocs dans le système de fichiers.
  3. La liste des blocs libres (Bit Vector).
  4. L'emplacement du répertoire racine.
  5. La date et l'heure de la dernière modification du SF,
  6. Une information indiquant s'il faut tester l'intégrité du SF.

# Implémentation du Système de fichier

## **Exercice 01:**

Que risque-t-on si le super-bloc est corrompu, et quelle est la solution ?

## **Réponse:**

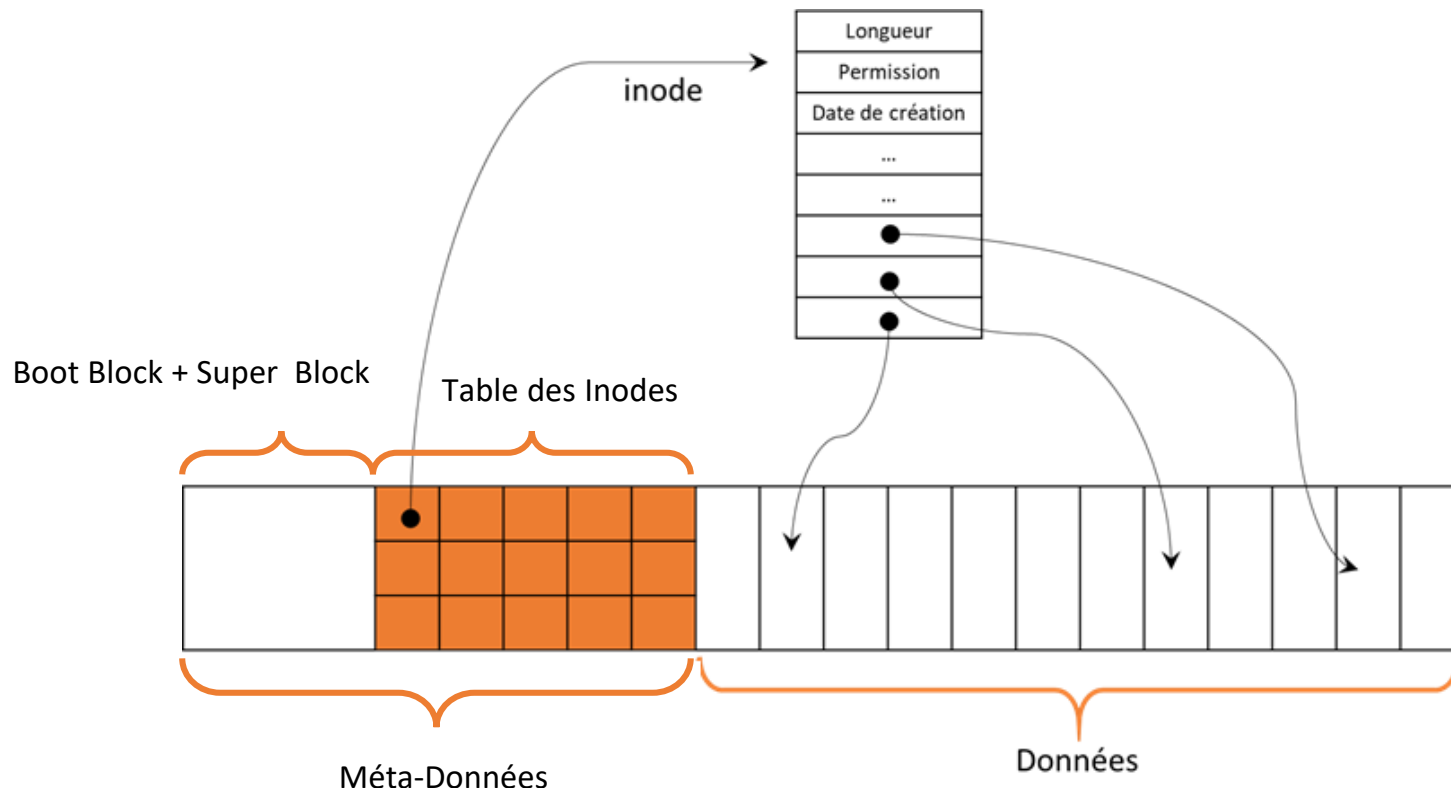
Le système de fichiers risque de ne plus pouvoir accéder aux fichiers. Pour réduire le risque, il maintient plusieurs copies du super-bloc. Ces copies permettent de vérifier l'intégrité du super-bloc et de le restaurer.

# Plan

- 1) Organisation des données sur le disque
- 2) Notion de fichier
- 3) Types de fichier
- 4) Système de fichier
- 5) Implémentation du Système de fichier
- 6) Notion d'i-node**

# I-node

- L'**inode** est une structure de données (une table d'index) héritée des premiers systèmes Unix (années 50).
- Elle stocke les **attributs essentiels d'un fichier** (permissions, taille, dates, pointeurs vers les blocs de données, etc.) **à l'exception de son nom**.



# I-node

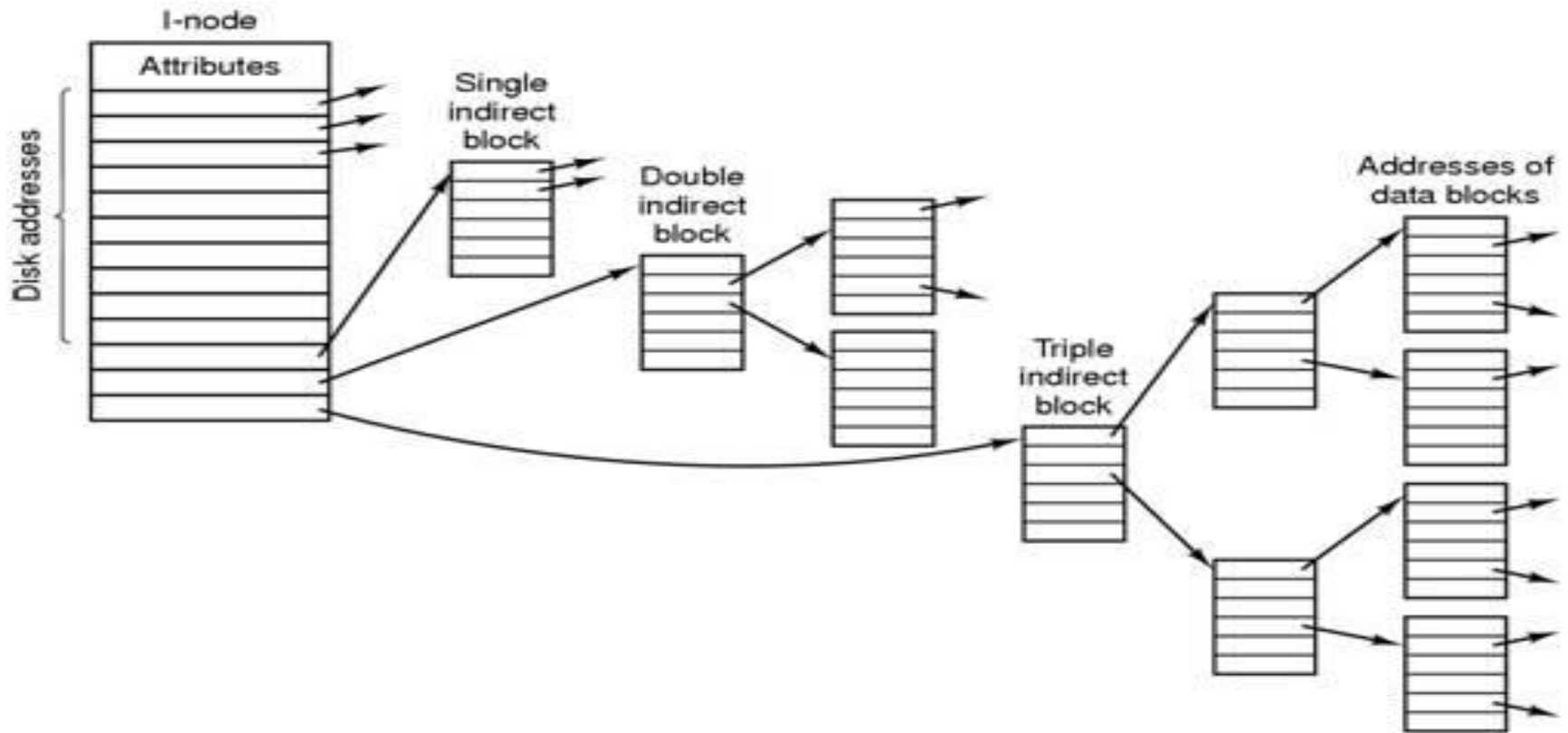
- Le système Linux EXT2/EXT3 répertorie chaque fichier par un numéro unique pour tout un disque.
- À chaque numéro correspond un **i-node**, comportant un nombre fixe de champs.
- Certains de ces champs, 13 au total, mémorisent l'emplacement physique du fichier.
- Les 10 premiers champs (sur les 13) contiennent les adresses des 10 premiers blocs composant le fichier.

| Attributs du fichier |
|----------------------|
| Adresse bloc n°1     |
| Adresse bloc n°2     |
| Adresse bloc n°3     |
| ...                  |
| Adresse bloc n°10    |
| Bloc n°11            |
| Bloc n°12            |
| Bloc n°13            |

# I-node

Pour les fichiers de plus de 10 blocs, on a recours à des indirections. Dans ce cas, le bloc n° 11 contient le numéro d'un bloc composé lui-même d'adresses de blocs de données.

Si ce n'est pas assez, de la même manière, le bloc n° 12 contient une adresse, un pointeur, à double indirection, et, si ce n'est pas encore assez, le bloc n° 13, un pointeur à triple indirection.



# I-node

## Remarques:

- Des éléments contigus à l'intérieur du fichier ne sont pas forcément contigus sur le disque lui-même => c'est le problème de la **fragmentation**,
- Le **bit Vector** qui se trouve dans le **super Block** informe pour chaque bloc de la zone des données s'il est libre ou pas, pour cela chaque bit du vecteur est positionnée à 1 pour dire que le bloc est utilisé, sinon 0.

